07-3-log-redaction

accounts-api security assessment · generated from the repository, not hand-edited

> **Section 7 deliverable: what is redacted, at which layer, and what the redaction still leaks.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/06-log-redaction.md`

Log redaction — what is removed, where, and what still leaks

Threat **T5**. This is the exfiltration path that needs no exploit: one `log.Info("request", "body", req)` in a hot path and the customer ledger is in a log aggregator with a hundred readers and a different retention policy.

Honesty is the point of this page. A redaction spec that claims completeness is worse than none, because it stops people looking.

Data in scope

field	classification	in logs?
name, email, phone	PII (DPDP Act)	**never**
IBAN	PII + payment instrument (RBI)	**never**
card last-4	cardholder data (PCI DSS scope)	**never** — see the note below on why last-4 is treated as in-scope here
balance, transaction amounts	financial PII	**never**
account id (internal UUID)	pseudonymous identifier	yes — this is the join key that makes logs useful
customer id (internal UUID)	pseudonymous identifier	yes
request id, trace id, route, status, latency	operational	yes

Card last-4 is not cardholder data under a strict PCI reading (it is a permitted truncation). I treat it as in-scope anyway, because last-4 plus name plus phone — all three of which pass through this service — is a working social-engineering kit against the bank's own call centre, and that attack does not care what the standard says.

Layer 1 — the application (primary, and the only one I actually trust)

Redaction happens at **serialisation time**, in the logger, not by pattern-matching a formatted string afterwards. Concretely:

- The logging call site takes a typed struct, never `interface{}` and never a formatted message. Custom `MarshalLog` implementations emit an **allowlist** of fields. A new field added to `Account` next quarter is invisible to logs unless somebody adds it to the marshaller — the default is redaction, and it stays correct as the model changes.
- Sensitive types are their own Go types (`type IBAN string`) whose `String()` and `MarshalJSON()` return `[REDACTED:iban]`. Passing one to any formatter, anywhere, including a third-party library's error path, produces the redacted form. This is the single highest-leverage thing in this document: it makes the safe behaviour the *default* rather than a rule engineers must remember.
- Errors are wrapped with `%w` and a static message; the query string, the row values and the request body never enter an error.
- `LOG_REDACTION_MODE=strict` (set in the manifest) fails closed: an unknown field type is logged as `[REDACTED:unknown]` rather than as its value.
- A unit test asserts that marshalling a fully-populated `Account` produces a JSON document containing none of the fixture values — including the two published example IBANs that appear in `waivers/waivers.yaml` as a gitleaks false positive.

Why here rather than downstream: at this layer I know the *type* of every value. Every layer below sees only text, and text-based redaction is a guessing game.

Layer 2 — the log pipeline (defence in depth, and it is not a substitute)

Fluent Bit DaemonSet, before anything leaves the cluster:

- A `modify` /Lua filter drops known-bad keys (`password`, `authorization`, `set-cookie`, `iban`, `pan`, `card`) at any nesting depth.
- Regex scrubbers for IBAN (`[A-Z]{2}\d{2}[A-Z0-9]{11,30}`), PAN with a Luhn check, and e-mail. Deliberately **narrow patterns**: a broad `\d{13,19}` would eat transaction ids and latencies and produce logs nobody can debug with, which ends with the filter being disabled.
- Any record that trips a scrubber is counted in a metric and, above a threshold, raises a ticket against the owning service. **The metric is the real output of this layer** — it tells me the application layer has a hole, which is actionable, whereas silently scrubbing tells me nothing and lets the hole grow.

Layer 3 — storage and access

CloudWatch Logs in `ap-south-1` only, KMS-encrypted with a customer-managed key, 90-day retention for application logs and 400 days for audit records. Log-group access is a separate IAM role with a quarterly access review; the log-archive account holds the copy that matters, under Object Lock.

What this still leaks

In descending order of how much it bothers me:

1. **The KYC provider request/response path.** Fields sent to the vendor are, by necessity, unredacted — that is the point of a KYC call. They appear in the Istio egress access log (SNI, byte counts, timing — not bodies) and in the vendor's own logs, which are outside my control entirely. Field minimisation in the KYC call is the mitigation, and it is an application change I have not made.

2. **Database slow-query logs.** `log_statement = 'ddl'` in the parameter group keeps statements out, but `log_min_duration_statement` — which any DBA will want on during a performance incident — logs the *query text with bound parameters*, i.e. IBANs, into CloudWatch. There is no clean fix: the setting is genuinely needed. Mitigation is procedural (enable during an incident, disable after, alert on the parameter-group change) and procedural mitigations decay.
3. **Panics and stack traces.** A Go panic prints function arguments. If an `Account` value is on the stack, the redacting marshaller is not involved, because the runtime formats it. `panic` handlers that scrub exist but are a best-effort text pass — layer 2's weakness, applied to the worst case.
4. **APM and error-reporting agents.** Most capture the process environment, local variables and request context, and ship them to a third-party SaaS — outside both layers above, and often outside the region, which makes it a residency problem as well as a privacy one. This is a large part of why the manifest moved the DB credential out of the environment (finding A5): it removes the worst single item those agents would collect.
5. **Cardinality as an oracle.** Redaction preserves structure. `account_id` appears in logs by design, so anyone with log access can build a per-account activity timeline — when a customer logs in, how often, from which route. That is not "no PII", it is pseudonymous PII, and under DPDP it is still personal data. I keep it because logs without a join key are not usable for debugging; the compensating control is access control on the log group and the quarterly review, not redaction.
6. **The URL path.** `/v1/accounts/{uuid}` is fine. If a future endpoint takes an IBAN or an email as a path segment or query parameter, it lands in the ALB access log and the CloudFront log **before** any of the three layers above see it. Nothing here would catch that. The control is a review rule ("no PII in URLs") — and a review rule is not a control, which is why I would encode it as a Semgrep rule over the route definitions. Not built; named in `DECISIONS.md` §6.
7. **Metric labels.** Prometheus labels are logs by another name and go through none of this. A well-meaning `http_requests_total{customer_id=...}` is both a PII leak and a cardinality outage.

What I did *not* do, and why

I did not attempt log-level tokenisation or format-preserving encryption of PII so that logs stay joinable while being "safe". It is technically appealing and it moves the problem: you then hold a token-to-PII map with the same sensitivity as the original data, plus a new key-management story, plus a decryption service that becomes the highest-value target in the estate. For a service of this size the allowlist marshaller gets 95% of the benefit for 5% of the machinery. If the bank later needs cross-system joins on customer identity in a data lake, that is when tokenisation earns its cost — and that decision belongs to the data platform, not to this service.